

REVERSIBLE WORKFLOWS

Engineering implementation note / 01 — Compensating actions

I am Amin Azimi, AI Architect and developer of Aegis Agent. I use explicit compensating operations and bounded retry logic to make selected agent workflows easier to recover from. I describe what the current code actually does, including failure modes that prevent me from claiming guaranteed restoration.

01 | Transaction-like workflow lifecycle

I implement a ReversibleWorkflowManager with three main lifecycle controls: begin() starts an active workflow, commit() clears registered compensations and closes it, and rollback() attempts compensations in reverse registration order. This is an application-level compensation stack, not a database transaction spanning external systems.

02 | Registering compensation

My execute(action, rollback) method runs the action first. Only after the action returns successfully does it push its rollback callback onto the stack. If the action throws before registration, this method does not automatically undo its partial effects. The caller must explicitly handle such failures and any required cleanup.

03 | Reverse-order rollback

I process the stack last-in, first-out so later successful actions are compensated before earlier ones. The implementation catches exceptions raised by a compensation and continues attempting remaining callbacks. This improves best-effort cleanup but also means rollback() can finish without reporting incomplete restoration.

04 | Concurrency and durability

I use a threading lock around lifecycle and stack mutations. However, the callbacks are process-local Python callables, not durable recovery records. A process crash loses the stack. The implementation does not establish distributed transaction coordination, idempotent compensation, crash recovery, or safe parallel execution of an action with concurrent commit/rollback.



RETRY GUARDS & RECOVERY

Engineering implementation note / 02 — Failure containment

05 | Bounded retry execution

I implement RetryGuard with a configurable retry budget, defaulting to two retries after the initial attempt. The execute() wrapper calls the supplied action up to three times in total. Successful execution resets failure state; exhausted attempts raise a RuntimeError chained from the last exception.

06 | Per-task status and telemetry

I maintain in-memory status keyed by task: failure count, retry count, last error and basic telemetry. A failed attempt produces a degraded state while the retry budget remains; exhaustion marks the task as open. These are lightweight state labels, not a persistent observability service or a time-based circuit-breaker reset policy.

07 | Circuit-breaker interpretation

I deliberately distinguish a retry guard from a full circuit breaker. The should_retry() helper reports whether the failure count is within budget, but execute() does not first reject every new call based on an already-open status. Therefore I do not claim a comprehensive half-open/open/closed breaker protocol or cross-process protection.

08 | Integration boundary

The Aegis orchestration path instantiates the reversible manager and retry guard and exposes retry state. That integration does not, by itself, prove that every dispatched agent operation registers an effective compensation callback or that every failure invokes a complete rollback. Recovery guarantees must be assessed per action and per calling path.

09 | Evidence and engineering limits

My statements are grounded in core/reversible_workflow.py and core/retry_guard.py, with orchestration context from app/orchestrator.py. I have not represented source inspection as executed fault-injection testing. I do not claim exactly-once effects, durable sagas, guaranteed rollback, automatic incident repair or zero data loss.

ACTIVE DEVELOPMENT — I am continuing to strengthen compensation coverage, rollback failure reporting, idempotency and restart-safe recovery before presenting these controls as production-grade guarantees.